1 Introduction

Understanding if a machine learning model is making accurate predictions or simply guessing at random is not trivial.

Bayesian machine learning seems to provide the solution, in the article, a new gradient-free training algorithm based on Approximate Bayesian Computation by Subset Simulation is proposed. In the proposed method, the likelihood function and the weights are defined by non-parametric formulations.

The experiments, especially the engineering case study on composite materials subject to fatigue damage, show the ability of the proposed algorithm to consistently reach accurate predictions while avoiding gradient related instabilities, and most importantly, it provides a realistic and coherent quantification of the uncertainty represented by confidence bounds which will be illustrated in the figures bellow.

Little video tutorial on How to handle Uncertainty in Deep Learning

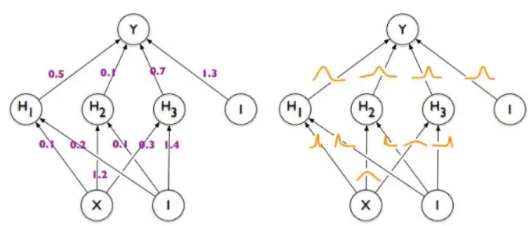
2 Background

2.1 FNN (Feedfoward Neural Network) and BNN (Bayesian Neural Network)

Let $D = (x, y) \in \mathcal{X} \times \mathcal{Y} \subseteq \mathbb{R}^m \times \mathbb{R}^l$ be the distribution of the dataset.

Let $\mathcal{M}$ be a model (here it is a FNN) with parameters $\theta = \{w, b\}$ (où w are the weights and b the biais).

BNN consists of esmitimating the a posteriori distribition $p(\theta|D, \mathcal{M})$ of θ assuming the model $\mathcal{M}$ and the data set D (whereas a standard FNN just gives one point estimation of θ). Here we have an FNN on the left and BNN on the right.



2.2 Approximate Bayesian computation by subset simulation

2.2.1 ABC (Approximate Bayesian Computation)

A feedforward pass of the FNN $\mathcal{M}$ with the input x and $\theta \sim p(\theta, \mathcal{M})$ gives $\hat{y} = f(x, \theta)$

The ABC algorithm consists at simulating samples of θ, with theses samples, generating samples of $\hat{y}$ and keeping the θ's whose respective $\hat{y}$ are close to y. And with theses samples estimate $p(\theta|D, \mathcal{M})$.

2.2.2 SS (Subset Simulation)

ABC is thus based on a rejection method, where we want the quantity $\rho(\eta(\hat{y}, \eta(y)))$ to get bellow a tolerance ϵ (ρ is a dissimilarity measure and η is a summarizing statistic).

The subset simulation method is used to reach tolerance faster. It is a iterative method where we begin with a high tolerance, we generate samples using a portion the previous batch(which has $\hat{y}$s closest to y) and shrinking the tolerance. We iterate until we get a tolerance smaller than ϵ.

Since each time we use the best samples to generate the next batch in the next iteration, we go faster.

3 Training BNN By ABC-SubSim

3.1 Proposed Methodology

First choose a sample size N, $p_0 \in [0, 1]$ such that $p_0 \times N$ and $p_0^{-1} \in \mathbb{Z}^*$

1. Generate N samples of $\theta = \{w, b\}$ from it's prior user defined PDF $p(\theta|\mathcal{M})$
2. Use them to run a forward pass and obtain N outputs $\hat{y}(\theta)$.
3. Evaluate the metric $\rho(\eta(\hat{y}), \eta(y))$ for each sample $\theta, \hat{y}(\theta)$.
4. Keep $p_0 \times N$ θ with the lowest metric value ρ as seeds.
5. These seeds are used to:

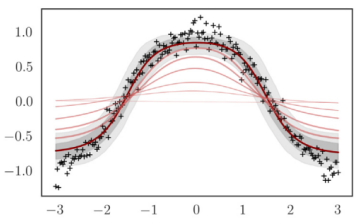
- automatically fix the tolerance value ϵ_j, as the highest metric value ρ among the seeds.
- obtain p_0^{-1} new samples from each seed verifying $\rho \leq \epsilon_j$ using the Modified Metropolis Algorithm (MMA).

6. repeat the process until $\epsilon_j \leq \epsilon$

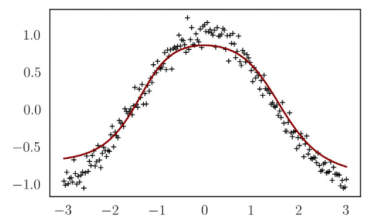
The distribution of parameters in the final subset constitute the marginalized posterior $p_\epsilon(\theta|D, \mathcal{M})$ whose information is used to produce robust predictions and quantify their uncertainty.

3.2 Illustrative Problems

The aim is to do estimate from noised data. $y = cos(x) + \zeta$ where $\zeta \sim \mathcal{N}(0.1)$

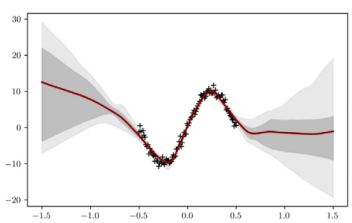


(a) *BNN by ABC-SS*

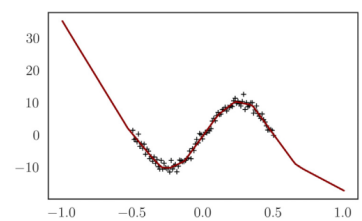


(b) FNN trained with Batch Gradient Descent

$y = 10sin(2\pi x) + \zeta$ where $\zeta \sim \mathcal{N}(0.1)$

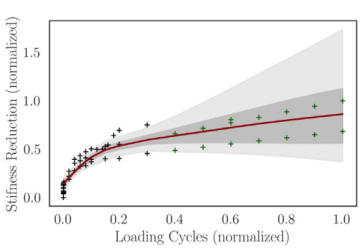


(a) *BNN by ABC-SS*

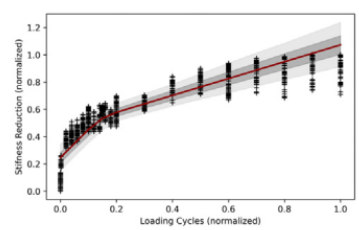


(b) FNN trained with Batch Gradient Descent

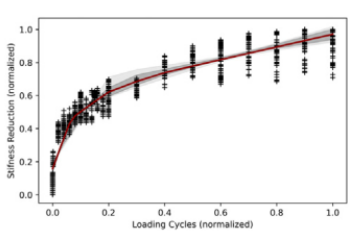
3.3 Case Study



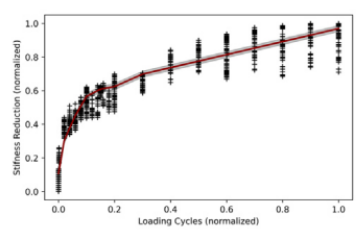
(b) Predictions of stiffness reduction.



(a) Variational Inference (Bayes by Backprop)



(b) Hamiltonian Monte Carlo



(c) Probabilistic Backpropagation